Homework 1: Neural Networks and CUDA Performance Analysis

Haritha Gottumukkala
Department of Applied Data Science
San José State University
San Jose, California, USA

TABLE I
PERSONAL PARAMETERS

Parameter	Value
SID4	1598
SEED	1598
SLICE	598
HP_ID	2
CLS_A	8
CLS_B	5

***Abstract*—In this homework, I worked with autoregressive models, neural networks, and CUDA programming. For the neural network part, I trained the same basic feed-forward network using both PyTorch and TensorFlow. I compared the required baseline model with the modified configuration assigned by my HP_ID. My HP_ID was 2, so the main change was increasing the learning rate from 0.001 to 0.003. I trained each configuration with three different seeds and compared the average test accuracies and loss curves. For the CUDA part, I implemented matrix multiplication using CUDA blocks and threads and compared its execution time with a CPU implementation. I tested three matrix sizes and also used `nvprof` to separate kernel execution time from data-transfer time. The results helped me see how both learning rate and parallel GPU execution can have a large effect on performance.**

***Index Terms*—neural networks, PyTorch, TensorFlow, CUDA, GPU, matrix multiplication, autoregressive models**

I. INTRODUCTION

This homework covered three different topics that helped me connect concepts from machine learning and parallel computing.

First, I studied autoregressive models and how previous observations can be used to predict future values. Next, I trained neural networks using PyTorch and TensorFlow and compared the required baseline model with my assigned modified model. Finally, I implemented matrix multiplication in CUDA and compared CPU and GPU performance.

For all of the experiments, I used the personal parameters calculated from my SID4 so that the results could be reproduced consistently.

II. PERSONAL PARAMETERS

The personal parameters I used for this homework are shown in Table I.

III. AUTOREGRESSIVE MODELS

An autoregressive model predicts the next value in a sequence by using values that appeared earlier in that same sequence. The basic idea is that the recent past can contain useful information about what may happen next.

A simple first-order autoregressive model can be written as

$$x_t = c + \phi x_{t-1} + \epsilon_t \tag{1}$$

where x_t is the value being predicted, x_{t-1} is the previous value, c is a constant, ϕ controls how strongly the previous value affects the current value, and ϵ_t represents random error.

One example is weather prediction. If the temperatures from the last few days are known, they can provide useful information for predicting the next day's temperature.

Another example is sales forecasting. A company can use previous daily or weekly sales to estimate future demand.

Autoregressive ideas can also be seen in financial time-series data, audio generation, and language models. In text generation, for example, the model predicts the next token based on the sequence of tokens that came before it.

IV. NEURAL NETWORK EXPERIMENT

A. Dataset and Preprocessing

For this part of the homework, I used the diabetes dataset. It contains eight numerical input features and one binary target variable. The dataset had 759 observations.

Before training the models, I checked the data for missing values and duplicate rows. I also examined the relationships between the features and looked at their distributions.

The data was divided into 70% training, 15% validation, and 15% testing data. I used `random_state = 1598` for the split.

An important part of the comparison was keeping this split fixed. I used the same training, validation, and testing data for both PyTorch and TensorFlow so that differences in the final results would come from the models rather than from different dataset splits.

Fig. 1 shows the correlation matrix that I generated while examining the dataset.

I also plotted the distribution of each input feature. This helped me get a better idea of how the values were spread before training the models.

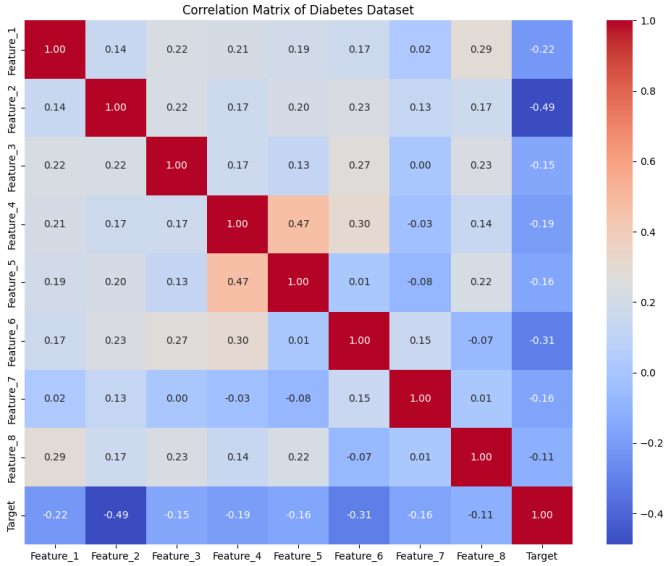

Fig. 1. Correlation matrix of the diabetes dataset.

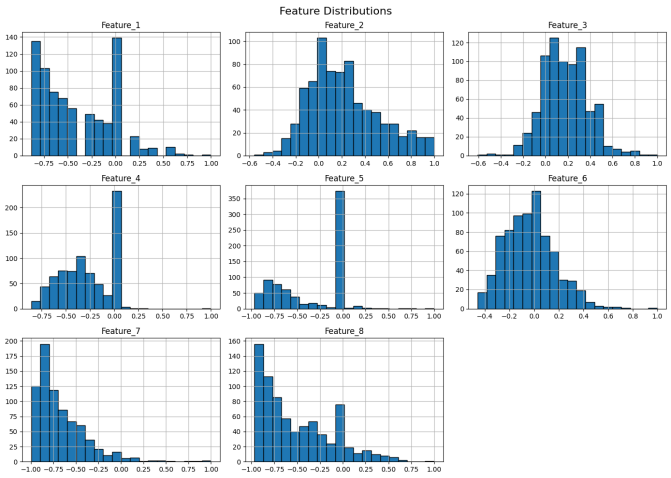

Fig. 2. Distribution of the input features in the diabetes dataset.

B. Baseline and Modified Models

The baseline neural network used two hidden layers with 64 and 32 units. The learning rate was 0.001 and the model was trained for 30 epochs.

My HP_ID was 2. According to the assignment, this kept the same hidden layers and the same number of epochs but changed the learning rate to 0.003.

Therefore, the two configurations were:

- Baseline: [64, 32], learning rate 0.001, 30 epochs.
- Modified: [64, 32], learning rate 0.003, 30 epochs.

I trained both configurations using seeds 1598, 1599, and 1600. The dataset split stayed fixed while only the training seed changed.

C. Overall Neural Network Results

The average results from the three seeds are shown in Table II.

TABLE II
THREE-SEED NEURAL NETWORK RESULTS

Framework	Model	LR	Mean Acc. (%)	Std. Dev.
PyTorch	Baseline	0.001	65.50	0.41
PyTorch	Modified	0.003	73.68	1.43
TensorFlow	Baseline	0.001	71.93	1.24
TensorFlow	Modified	0.003	73.10	0.83

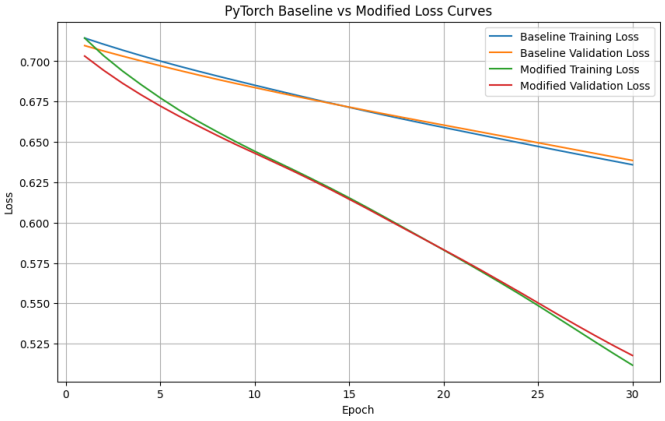

Fig. 3. PyTorch training and validation loss for the baseline and modified models.

D. PyTorch Results

The PyTorch results showed the clearest difference between the baseline and modified configurations.

The baseline model had an average test accuracy of 65.50%, while the modified model reached 73.68%. This was an improvement of about 8.19 percentage points.

For the run using seed 1598, the baseline test accuracy was 65.79% and the modified test accuracy was 75.44%.

When I looked at the baseline predictions, I noticed that it was predicting almost every test example as the majority class. Because of this, the accuracy alone did not give the complete picture of how well the model was learning. This behavior, together with the loss curves, suggested that the baseline model was underfitting.

Fig. 3 shows the training and validation losses for the seed 1598 run.

From the graph, I could see that the modified model learned faster than the baseline model. Its loss decreased more during the same 30 epochs. The training and validation losses also stayed reasonably close, so I did not see strong evidence of severe overfitting.

E. TensorFlow Results

The difference between the TensorFlow models was smaller.

The baseline model achieved an average test accuracy of 71.93%, while the modified model achieved 73.10%. This was an improvement of about 1.17 percentage points.

For seed 1598, both models produced the same test accuracy of 73.68%. However, when I looked at the loss curves, the modified model generally reduced its loss more quickly.

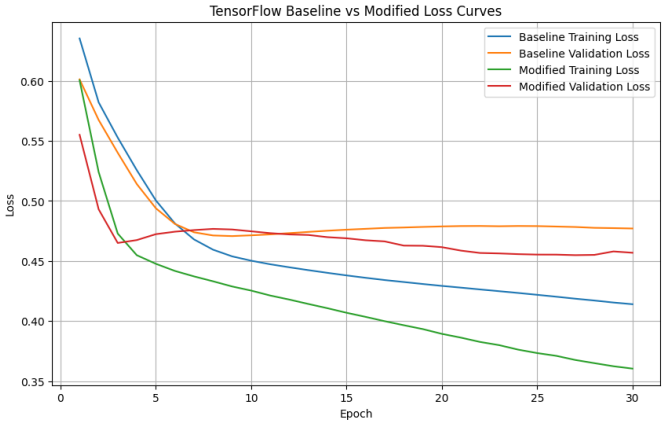

Fig. 4. TensorFlow training and validation loss for the baseline and modified models.

Toward the end of training, I noticed a small difference between the training and validation loss of the modified model. This could indicate a slight tendency toward overfitting, but the difference was not large enough for me to consider it severe.

F. Failure and Error Analysis

The clearest issue I observed was with the PyTorch baseline model. Although its test accuracy was 65.79% for the SEED = 1598 run, the model predicted almost every test example as the majority class. Because of this, accuracy alone made the model appear more useful than it actually was. Together with the relatively high training and validation losses, this behavior suggested that the baseline model was underfitting.

After increasing the learning rate from 0.001 to 0.003, the modified PyTorch model learned much faster. Its training and validation losses decreased substantially and remained reasonably close, so I did not observe strong overfitting in the modified model.

For TensorFlow, the difference between the two models was smaller. The modified model generally reached lower validation loss more quickly. Toward the end of training, I noticed a small gap between training and validation loss. This suggested a mild tendency toward overfitting, but the gap was not large enough for me to consider it severe.

These observations helped me understand why test accuracy should not be used by itself. Looking at the predictions together with the training and validation loss curves gave me a better picture of how each model was actually learning.

V. CUDA Matrix Multiplication

A. Blocks and Threads

The CUDA part helped me understand how GPU work is divided between blocks and threads.

I used a two-dimensional block containing

$$16 \times 16 = 256 \tag{2}$$

threads.

TABLE III
AVERAGE CUDA MATRIX MULTIPLICATION RESULTS

Size	CPU (ms)	Kernel (ms)	H2D+D2H (ms)	Speedup
256	2.829	0.113	0.397	5.546×
1024	214.517	4.794	4.907	22.113×
4096	20472.542	317.206	77.207	51.906×

Each thread was responsible for calculating one element of the output matrix C.

The row and column were calculated using

$$row = blockIdx.y \times blockDim.y + threadIdx.y \tag{3}$$

and

$$col = blockIdx.x \times blockDim.x + threadIdx.x. \tag{4}$$

After finding its row and column, the thread calculated the dot product needed for that position:

$$C_{ij} = \sum_{k=0}^{N-1} A_{ik} B_{kj}. \tag{5}$$

This means that many elements of the output matrix can be calculated at the same time instead of being calculated one after another on the CPU.

B. Experiment Setup

I ran the CUDA experiment on a Google Colab runtime with an NVIDIA Tesla T4 GPU.

The program was compiled for the `sm_75` architecture and used 16×16 threads per block.

I tested three matrix sizes:

$$256, \quad 1024, \quad 4096. \tag{6}$$

For every matrix size, I measured CPU execution time, GPU kernel time, host-to-device transfer time, device-to-host transfer time, and total end-to-end GPU execution time.

I also performed a warm-up kernel call before measuring the timed kernel. This was useful because the first CUDA operation can include initialization overhead that is not representative of normal execution.

Each matrix size was tested three times, and I used the averages for the final comparison.

C. Timing Results

The average CUDA timing results are shown in Table III.

The first thing I noticed was that the performance difference became much larger as the matrix size increased.

For the 256×256 matrix, the GPU was about 5.55 times faster than the CPU even when memory-transfer time was included.

For 1024×1024, the speedup increased to about 22.11 times.

The largest difference appeared for the 4096×4096 matrix. The CPU took approximately 20.47 seconds on average, while

TABLE IV
SELECTED NVPROF RESULTS FOR 1024×1024

GPU Activity	Time Share
Matrix multiplication kernel	83.23%
Device-to-host transfer	8.76%
Host-to-device transfer	8.01%

the GPU provided an end-to-end speedup of approximately 51.91 times.

D. Profiler Results

I used `nvprof` to check how the GPU execution time was divided between computation and memory transfers.

I profiled the 1024×1024 matrix case. The most important values from the profiler are shown in Table IV.

The profiler showed that most of the GPU activity was spent doing the actual matrix multiplication. The kernel accounted for about 83.23% of the GPU activity.

I also noticed that the profiler reported two kernel calls. This happened because my program first launched a warm-up kernel and then launched the kernel that I measured.

E. CPU and GPU Crossover

Among the matrix sizes I tested, 256×256 was already large enough for the GPU to be faster than the CPU after including transfer time.

The crossover does not happen at size zero because using the GPU itself has overhead. The matrices first have to be copied from CPU memory to GPU memory, the CUDA kernel has to be launched, and the result has to be copied back.

For a very small problem, this overhead can take a noticeable amount of the total execution time. As the matrix becomes larger, the amount of computation increases much faster, and the GPU has enough work to take advantage of its parallel threads.

This is what I could see in my results. The speedup increased from about 5.55 times for size 256 to about 22.11 times for size 1024 and finally about 51.91 times for size 4096.

VI. CONCLUSION

This homework helped me understand the topics better because I was able to compare the concepts with actual results instead of only looking at the theory.

In the neural network experiment, increasing the learning rate from 0.001 to 0.003 improved the average accuracy in both PyTorch and TensorFlow. The improvement was much larger for PyTorch, where the baseline model appeared to be learning too slowly during the 30 epochs.

The CUDA experiment made the idea of parallel execution much clearer to me. Each GPU thread calculated one part of the output matrix, allowing many calculations to happen at the same time. The timing results showed that this advantage became much larger as the matrix size increased.

The profiler results also helped me separate the time spent doing the actual calculation from the time spent moving data between the CPU and GPU. Overall, these experiments gave me a better understanding of both neural-network training and GPU-based parallel computing.